

Technical Report

I have moved away from the test agents (Alice, Faber, and Acme) that I used to start via the command `LEDGER_URL=http://test.bcovrin.vonx.io ./run_demo [agent_name]`. For simplicity, these agents relied on a Python script that automatically created a wallet, self-registered on Bcovrin, and wrote its schema (accessing the Blockchain), before generating the mock text interface I showed in our last team call.

To use the pure Hyperledger Aries engine and have full control over the flow, I created an isolated Python virtual environment to avoid conflicts with other system files. Then, I executed the following:

```
sudo apt update && sudo apt install python3.12-venv -y
python3 -m venv aries_env
source aries_env/bin/activate
pip install aries-cloudagent
```

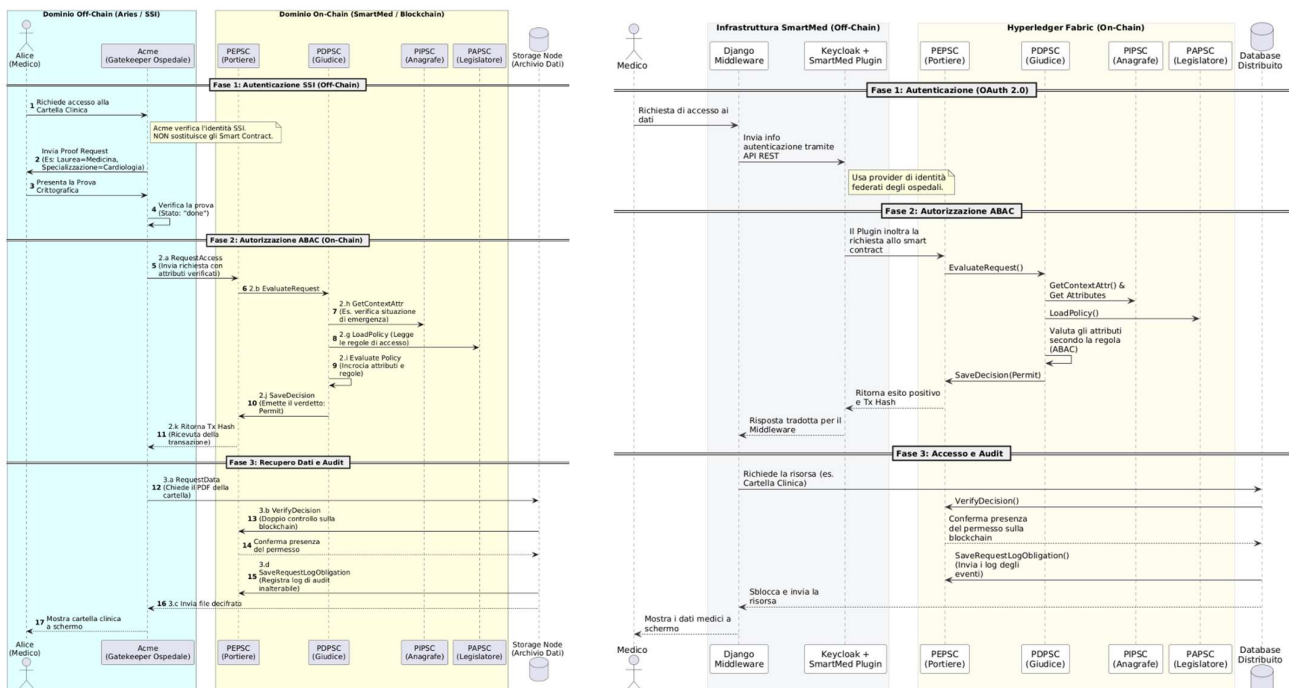
To start an agent, you must always first enter the virtual environment using the command:
`source aries_env/bin/activate`

Next, you need to run the specific startup files for each agent, which are provided in the attached "2_Agent_Startup_Scripts" folder.

I have created three agents, one for each fundamental role in the Decentralized Identity world:

- Issuer: MinistryOfHealth
- Holder: MedicalProfessional
- Verifier: SmartMedGateway

As shown in the following sequence diagram, I aimed to implement a specific use case: a physician requesting access to a patient's electronic health record.



In the current SmartMed system, if a physician wants to access a patient's records, they must verify their identity centrally, typically by entering credentials into the hospital's server (as shown in the left diagram).

In the new system utilizing Sovereign Identity, every physician has their own wallet where they store their own attributes (e.g., medical degree, valid employment contract, etc.) issued by certifying authorities, such as the Ministry of Health. If a physician wants to access a patient's record, they must demonstrate that they possess specific valid attributes.

My task was to introduce the role of the **Verifier**, which must ensure that the attributes presented in the request are valid and signed with the private key of the issuing entity (a mathematical verification performed by the agent using the entity's public key registered on the blockchain).

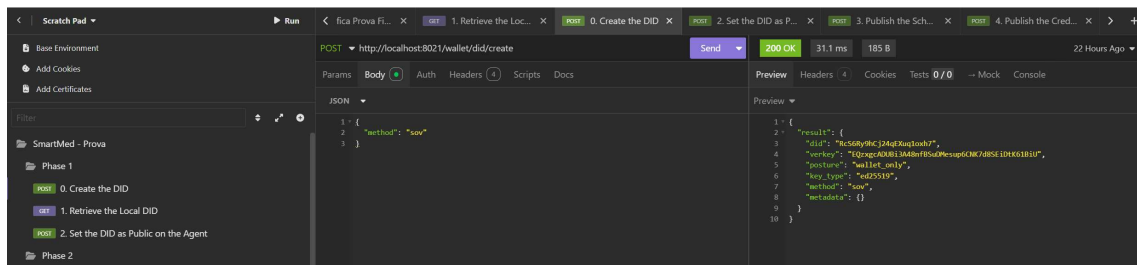
To implement this proof of concept, the first entity (or agent) I created was the Issuer, which in our case is the Ministry of Health, using the following script:

```
aca-py start \  
  -label "MinistryOfHealth" \  
  -inbound-transport http 0.0.0.0 8020 \  
  -outbound-transport http \  
  -endpoint http://localhost:8020 \  
  -admin 0.0.0.0 8021 \  
  -admin-insecure-mode \  
  -wallet-type askar \  
  -wallet-name MinistryWallet_SmartMed \  
  -wallet-key MinistrySecretKey123 \  
  -genesis-url http://test.bcovrin.vonx.io/genesis \  
  -auto-provision \  
  -auto-accept-invites \  
  -auto-accept-requests \  
  -auto-respond-credential-proposal \  
  -auto-respond-credential-request
```

NOTE: This agent, like the others used for testing, automatically accepts and responds to requests (thanks to flags such as `--auto-accept-requests` and `--auto-respond-credential-request`). In a real-world implementation, this process will not be automatic; instead, it must be managed via a Controller (backend logic) or a User Interface (UI) that allows the enterprise system or a human operator to securely validate, approve, or reject requests.

Once the agent was started from the terminal, and lacking a graphical interface, I used **Insomnia** to interact directly with its REST APIs and make my requests:

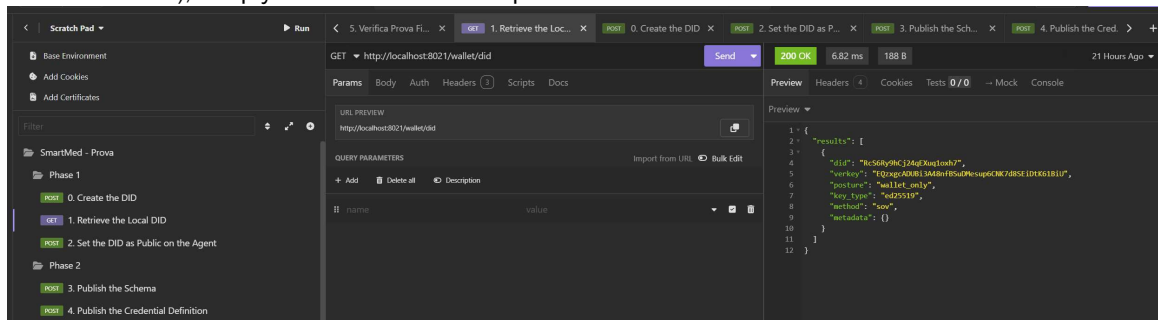
1. To enable the Ministry to officially create and sign a medical license, it must first be "registered" on the blockchain. To do this, we use the values of the "DID" (the identifier) and "Verkey" (the public verification key) attributes that were just generated and are present in its wallet.



- 1.1 To physically write this identity onto the public Ledger, you must use the official testnet website (<http://test.bcovrin.vonx.io/>) and fill out the registration form by entering the DID and Verkey.

The form is titled 'Authenticate a New DID' and includes the instruction 'Easily write a new DID to the ledger for new identity owners.' It features two radio buttons: 'Register from seed' (unselected) and 'Register from DID' (selected). Below these are input fields for 'DID', 'Verkey', and 'Alias (optional)'. A 'Role' dropdown menu is set to 'Endorser'. At the bottom is a blue button labeled 'Register DID' with a right-pointing arrow icon.

- 1.2 To verify that the DID has been successfully saved and configured in the Ministry agent's wallet (the local database), simply execute this GET request via Insomnia:



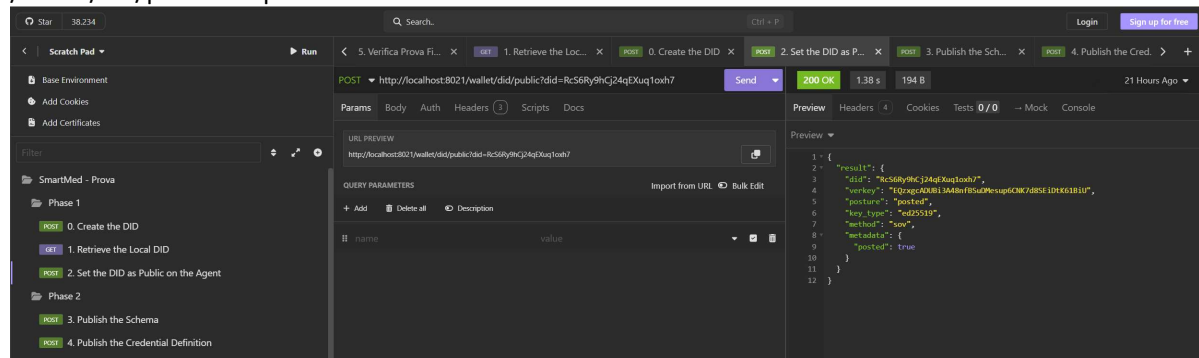
2. An Aries agent's wallet can contain several DIDs. However, to perform public and official operations on the Blockchain (such as publishing a Medical License Schema), the agent must elect a single DID as its official Public identifier.

In Hyperledger Aries, the word *posture* literally indicates the attitude or registration status that a specific DID has towards the outside world (the Blockchain).

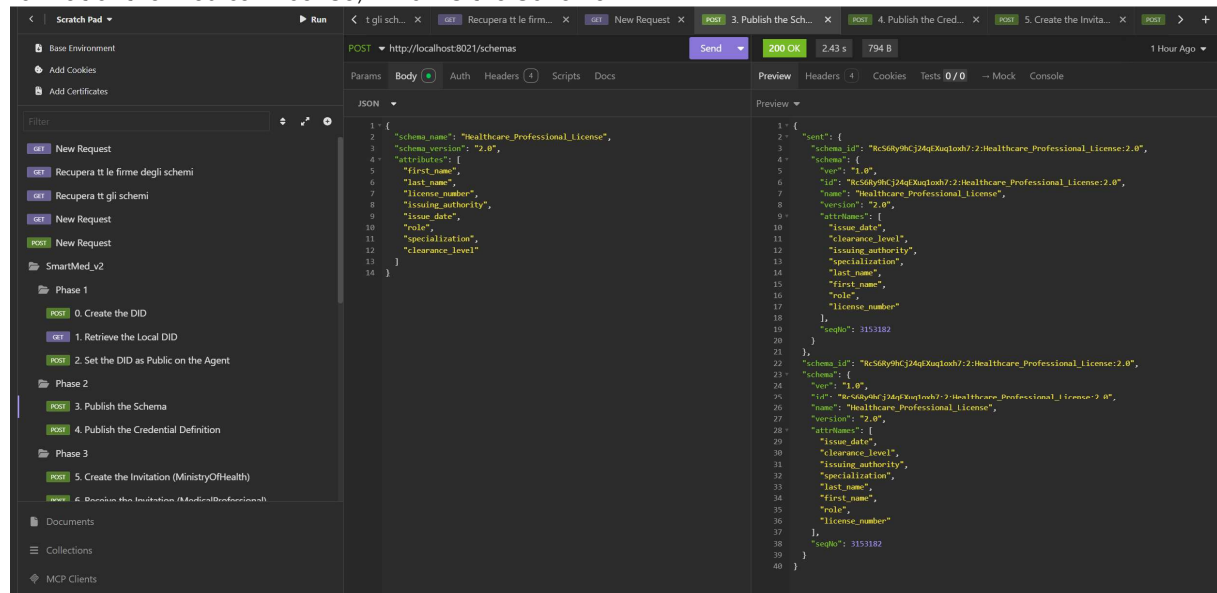
The 3 possible "postures" of a DID are:

- "posture": "wallet_only" (Wallet only). The DID has been generated and exists only locally in the agent's database.
- "posture": "posted" (Posted). This posture appears after the DID has been successfully sent and anchored on the Blockchain (in our case, Bcovrin).
- "posture": "public" (Public and Official). The DID is designated by the agent as its default public identity.

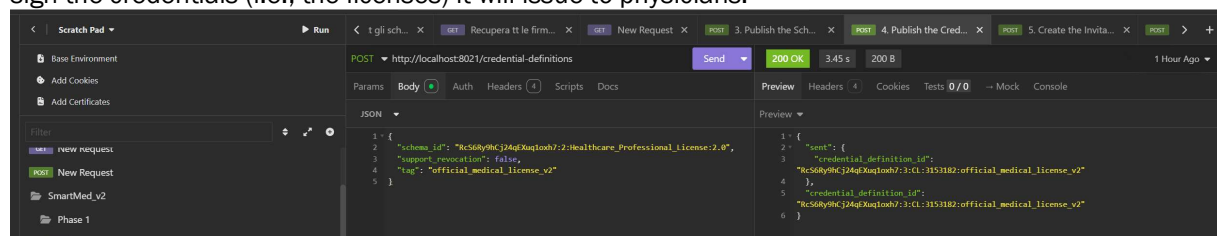
To promote a DID to an official public identity, it is necessary to make a POST request to the `/wallet/did/public` endpoint:



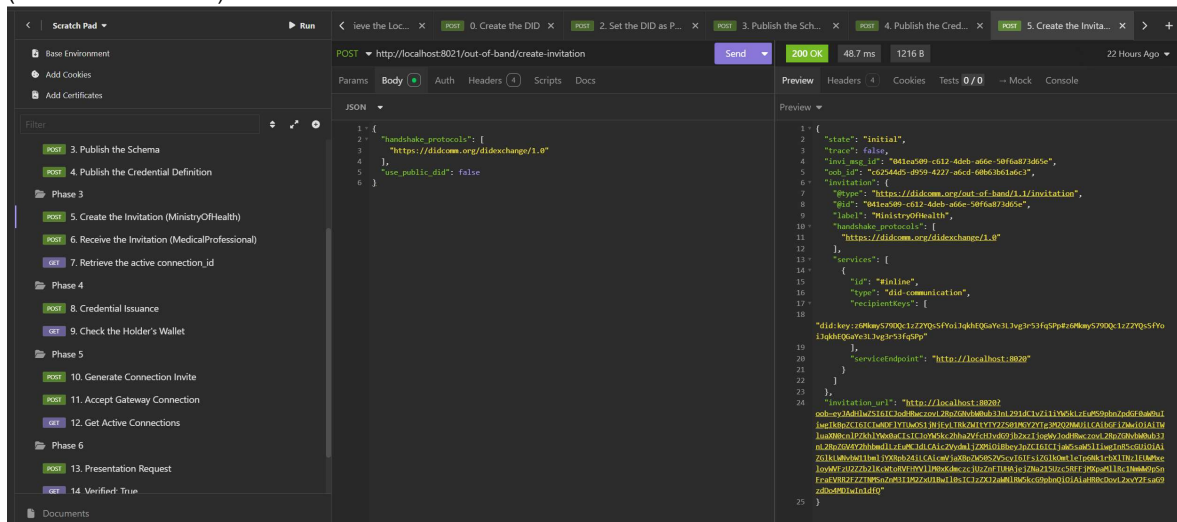
- Now that the MinistryOfHealth is a public entity verified on the Blockchain, it must register the official format of the Medical License, which is the Schema.



- Next, we create the cryptographic keys (the **Credential Definition**) that the Ministry will use to digitally sign the credentials (i.e., the licenses) it will issue to physicians.

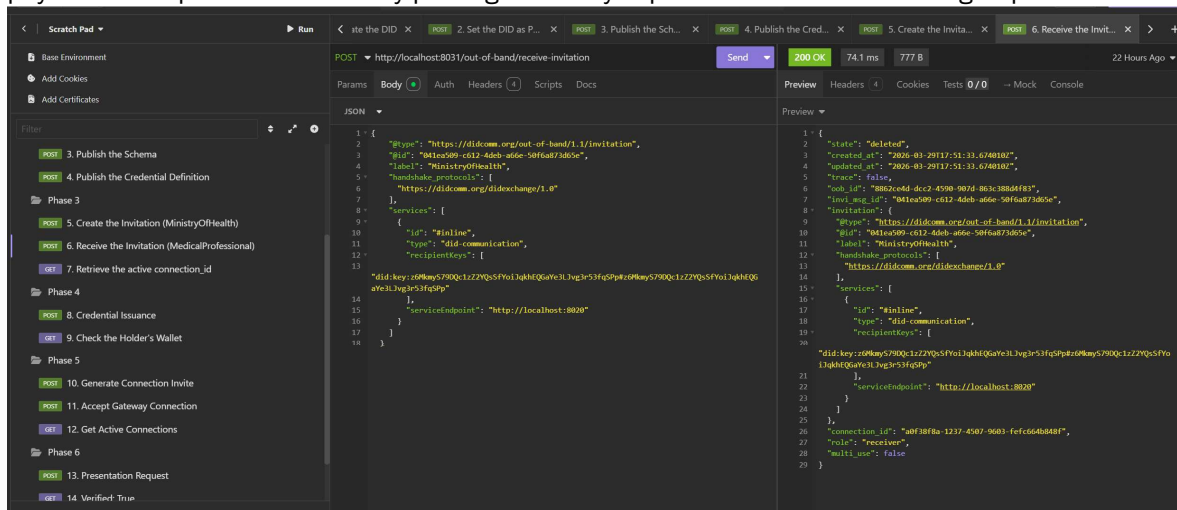


- At this point, we begin to connect the Ministry to the physician by generating an official invitation (Connection Invite):

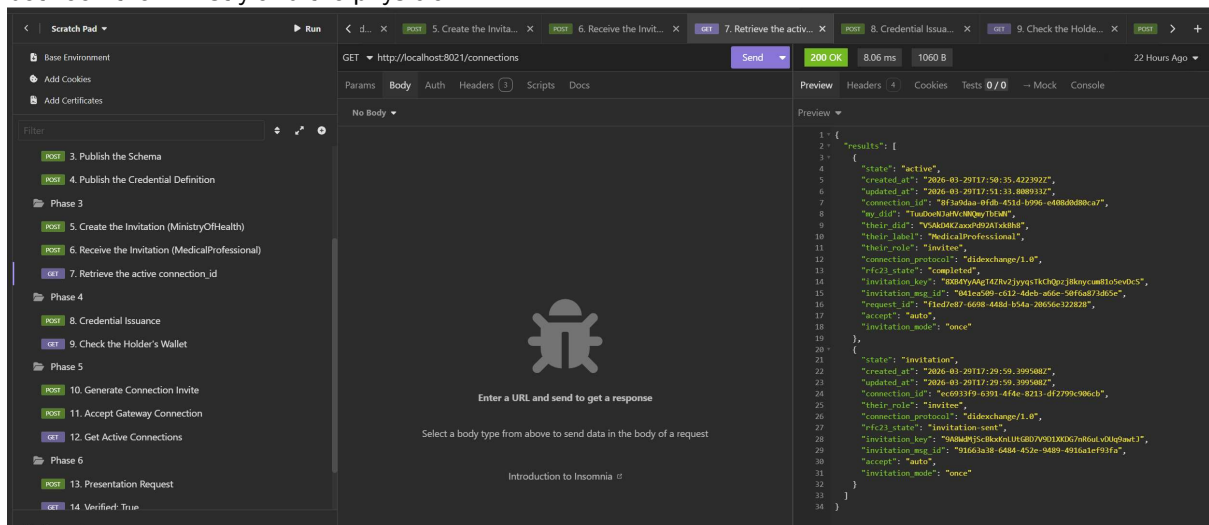


- From the generated response, you must copy all the content located inside the curly brackets {...} of the "invitation" field.

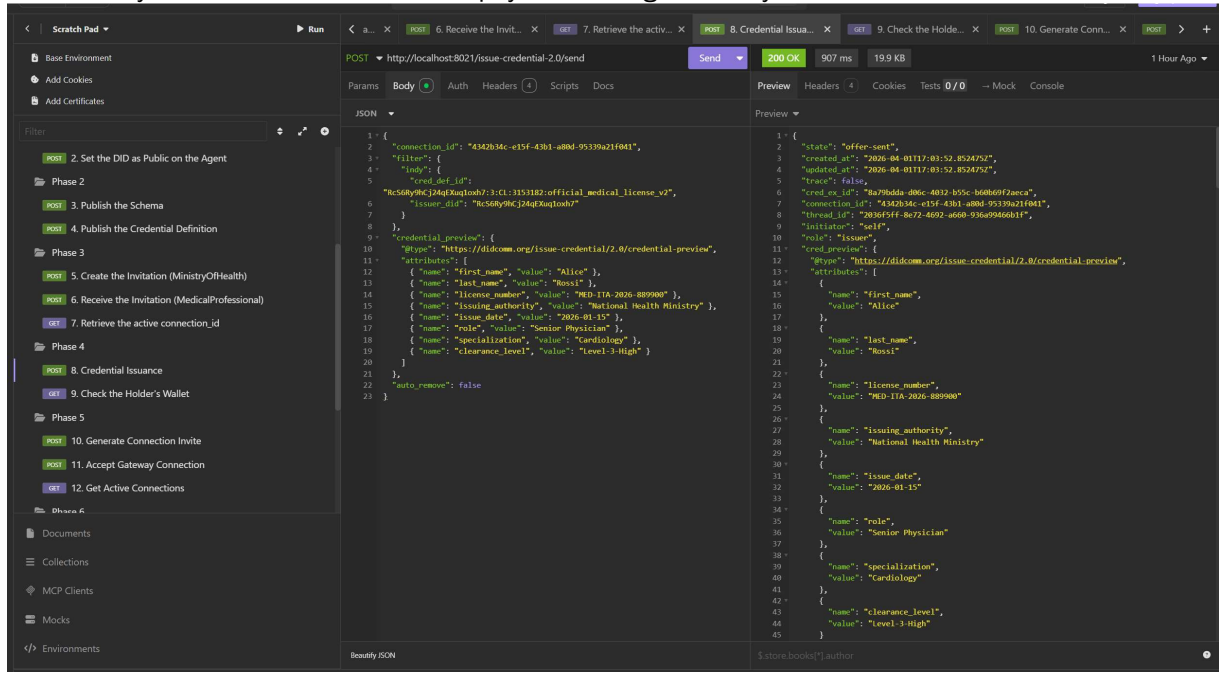
6. After starting the physician's agent (Medical Professional) via its script, we must ensure that the physician accepts the invitation by pasting the newly copied JSON into the following request:



7. Finally, in order to send messages, we must query the agent to retrieve the value of the "connection_id" field, which represents the unique identifier of the secure communication channel just established between the Ministry and the physician.

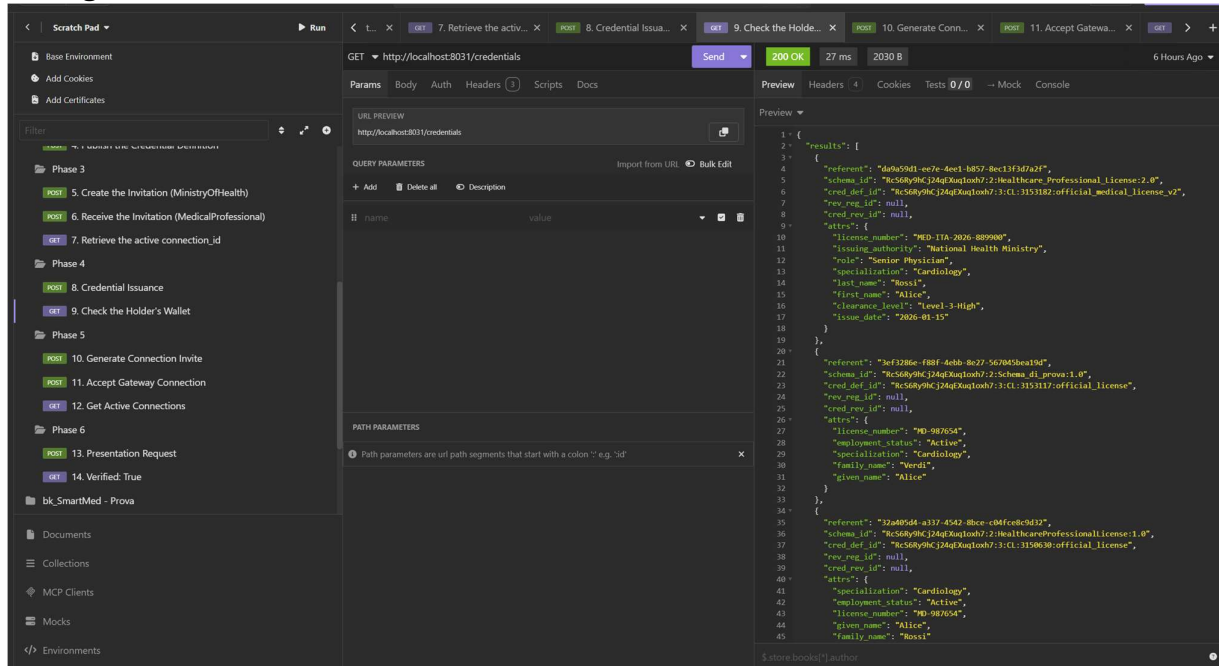


8. The Ministry issues the credential to the physician using the newly created connection:



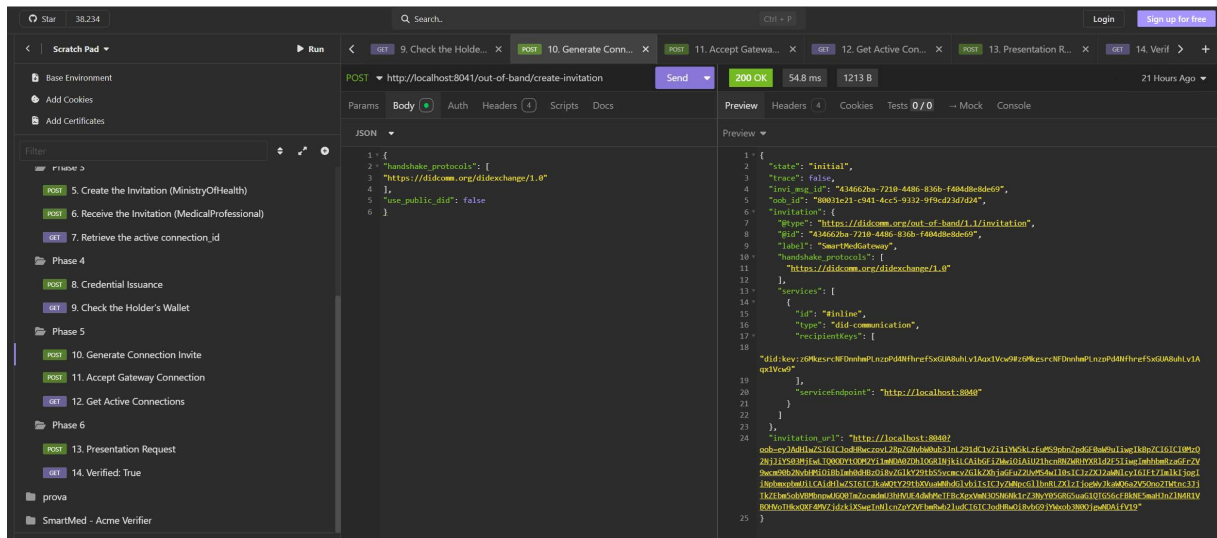
- NOTE: In the image, the `connection_id` field might appear different from the previous steps because I ran multiple tests to refine the attributes and test various connections.

9. To verify whether the physician has successfully received and saved the license in their wallet, we query their agent:

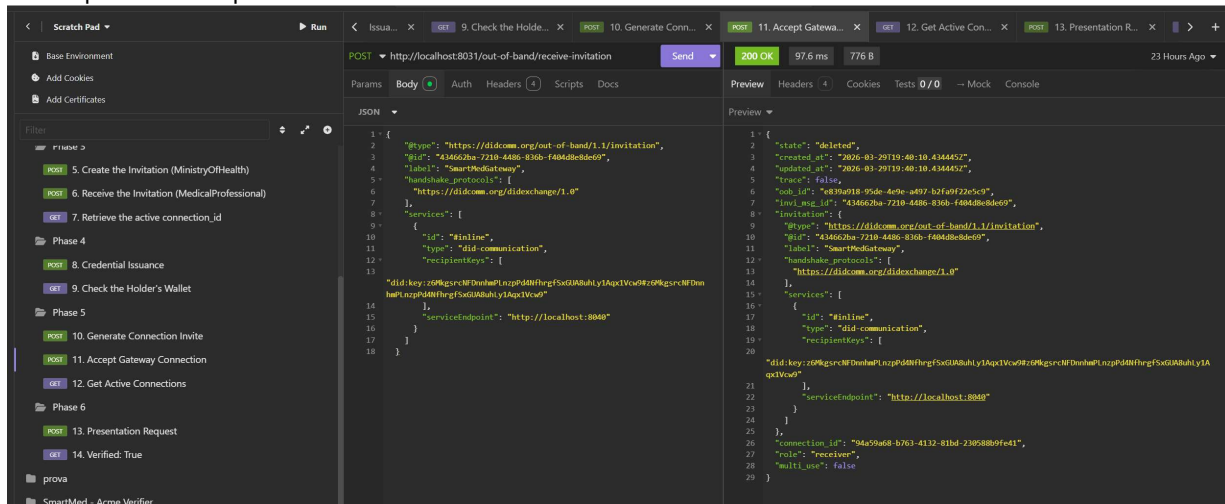


- NOTE: Since we started the physician's agent with auto-accept parameters for testing purposes, their wallet has already received the offer, accepted it automatically, and saved the certificate

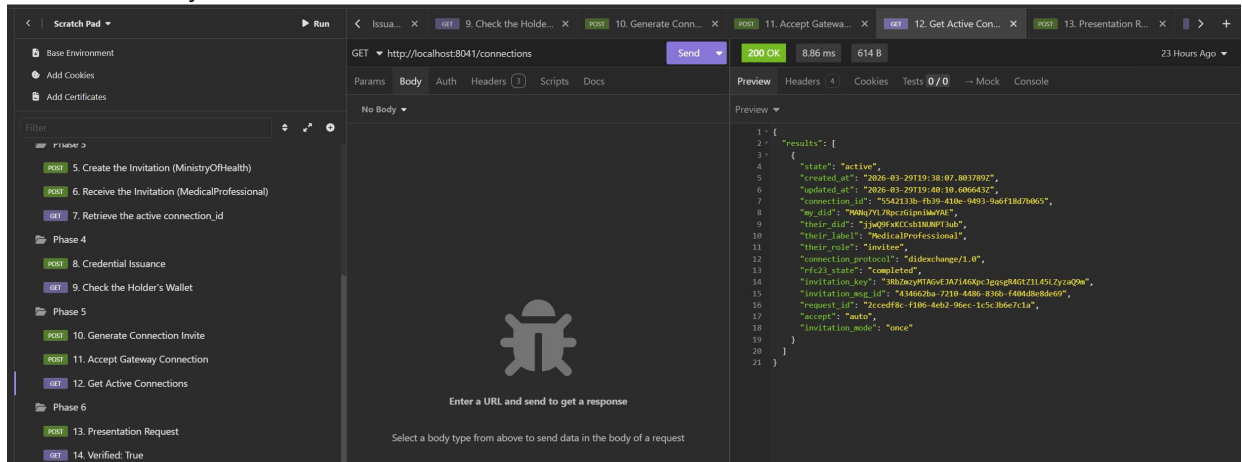
10. To activate the Verifier (the Hospital or Gateway), I also used a dedicated script in this case: `start_gateway.sh`. After starting it, we must connect it to the physician so the hospital can ask them to present their medical degree. Exactly as done previously for the Ministry, the Verifier must create a formal invitation, and the content of the "invitation": `{...}` field must be copied from the response.



11. The physician must accept the connection by pasting the invitation JSON (just copied) into the body of the request to accept the invite:



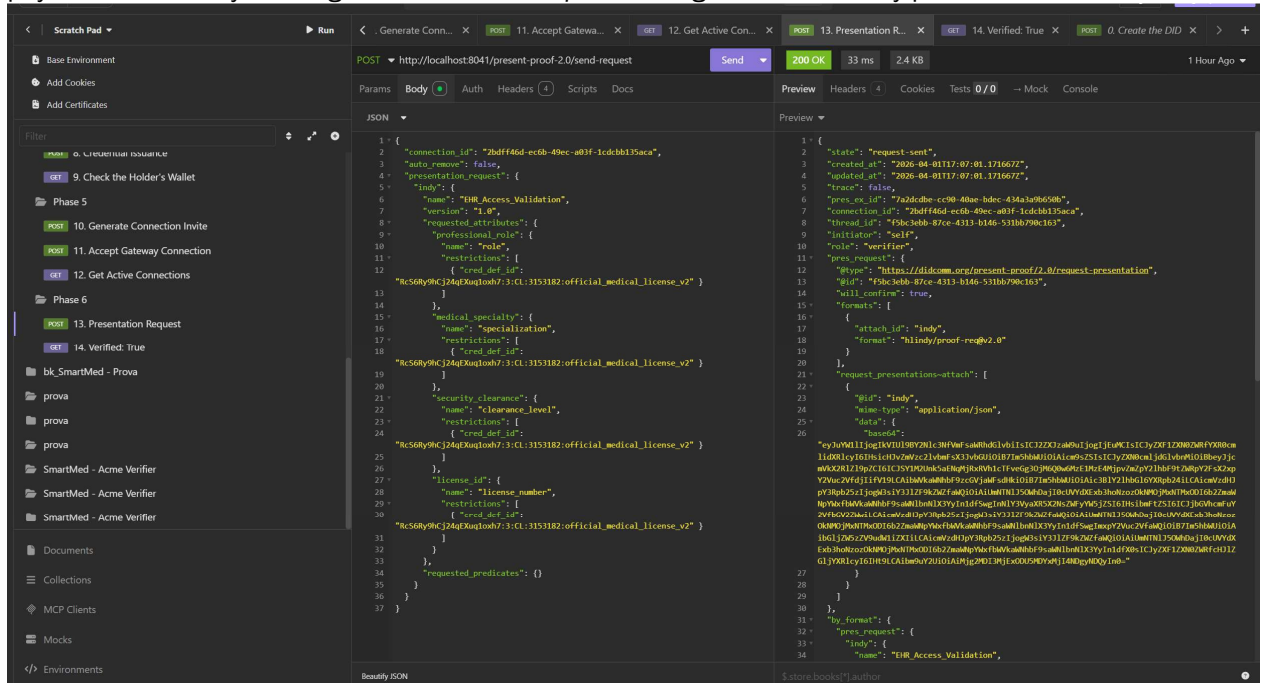
12. Once done, we retrieve the new connection_id relating to the secure communication channel just opened between the Physician and the Verifier.



The screenshot shows a REST client interface with a GET request to `http://localhost:8041/connections`. The response is a JSON object with the following structure:

```
1: {
2:   "results": [
3:     {
4:       "state": "active",
5:       "created_at": "2026-04-29T19:38:07.803789Z",
6:       "updated_at": "2026-04-29T19:40:38.686643Z",
7:       "connection_id": "55a23230-fb39-440e-9493-9edf18d7065",
8:       "my_did": "WbWq7Vh79pcd5ipmhwAE",
9:       "their_did": "ijwQ9wKCCabIMNP13ub",
10:      "their_label": "MedicalProfessional",
11:      "their_role": "Verifier",
12:      "connection_protocol": "didexchange/1.0",
13:      "rfc31_state": "completed",
14:      "invitation_key": "38b29p4fH0v1404qc3etp9M2114NzYzeQw",
15:      "invitation_wac_id": "434662ba-7210-4486-810b-f40d8d8d69",
16:      "request_id": "Xccedf8c-f106-4eb2-96ec-1c5c3bde7c1a",
17:      "accept": "auto",
18:      "invitation_mode": "once"
19:     }
20:   ]
21: }
```

13. At this point, we implement the real use case: when the physician requests access to a patient's medical records, even before their request is evaluated by the Smart Contracts, the Verifier (Gateway) queries the physician's wallet by sending a *Presentation Request* asking them to securely present their certificate.

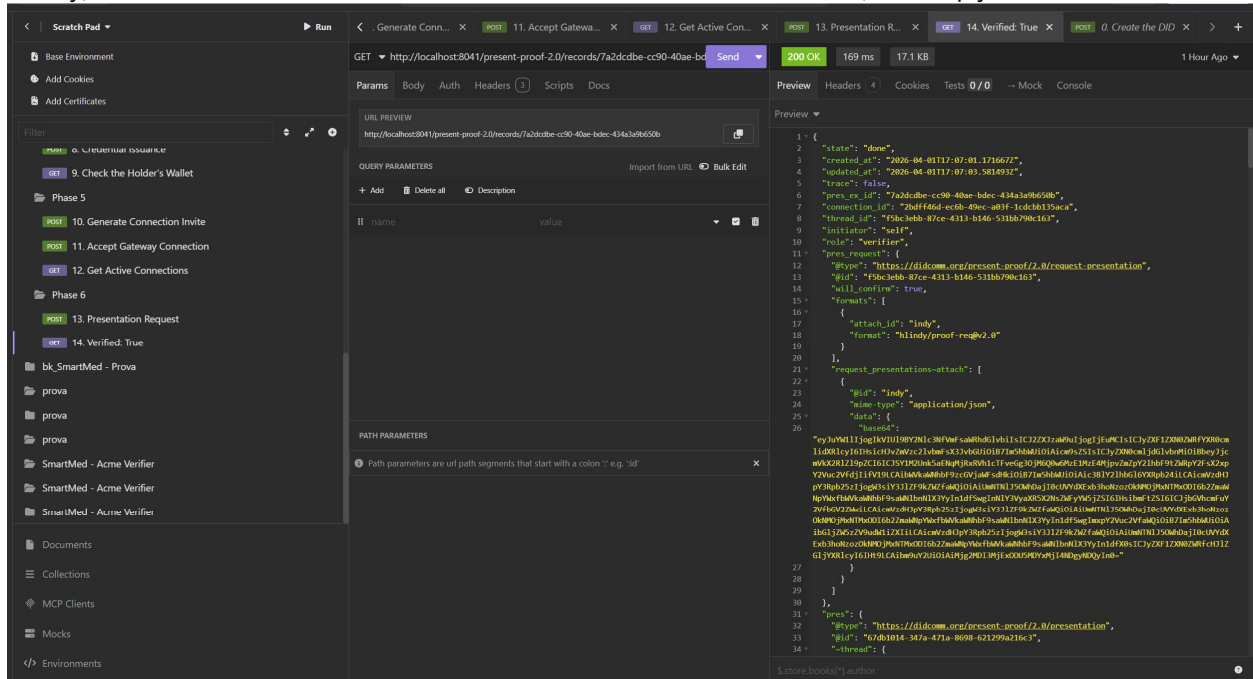


The screenshot shows a REST client interface with a POST request to `http://localhost:8041/present-proof-2.0/send-request`. The response is a JSON object with the following structure:

```
1: {
2:   "connection_id": "72bfff46d-ecdb-49ec-a03f-1cdcb135aca",
3:   "auto_remove": false,
4:   "presentation_request": {
5:     "indy": {
6:       "name": "EDR_Access_Validation",
7:       "version": "1.0",
8:       "requested_attributes": {
9:         "professional_role": {
10:           "name": "role",
11:           "restrictions": [
12:             { "cred_def_id": "Rc56hy9KJ24gQxq1m0v7:3:CL:3153182:official_medical_license_v2" }
13:           ],
14:           "medical_specify": {
15:             "name": "specialization",
16:             "restrictions": [
17:             { "cred_def_id": "Rc56hy9KJ24gQxq1m0v7:3:CL:3153182:official_medical_license_v2" }
18:           ],
19:           "security_clearance": {
20:             "name": "clearance_level",
21:             "restrictions": [
22:             { "cred_def_id": "Rc56hy9KJ24gQxq1m0v7:3:CL:3153182:official_medical_license_v2" }
23:           ],
24:           "license_id": {
25:             "name": "license_number",
26:             "restrictions": [
27:             { "cred_def_id": "Rc56hy9KJ24gQxq1m0v7:3:CL:3153182:official_medical_license_v2" }
28:           ],
29:           "requested_predicates": {}
30:         }
31:       }
32:     }
33:   }
34: }
```

- NOTE: In the request payload, I included the "auto_remove": false command. Without it, we would not see the result in the subsequent GET request. This happens because, in strict compliance with the Data Minimization principle, the Aries protocol immediately deletes the record from the database as soon as the cryptographic proof is successful

14. Finally, to check the verification outcome and extract the useful attributes, we simply make this last call



- NOTE: The "verified": "true" parameter represents the success of the entire operation. This value certifies that the Gateway has successfully verified the cryptographic proof (Verifiable Presentation) and has mathematical certainty that the certificate presented by the physician is authentic, unaltered, and was indeed signed with the Ministry of Health's private key.

In practice, through this process, I have implemented the AUTHENTICATION phase that takes place off-chain.

However, as requested by my colleagues, we need to ensure that the verifying agent also handles attributes, because a part of the AUTHORIZATION process actually begins off-chain. In fact, the Gateway must be able to extract the verified attributes (e.g., specialization, role) directly from the cryptographic proof, and then send them to the PEPSC contract on the blockchain.

To properly manage the attributes, they must be conceptually divided into two categories:

- **Static Attributes:** These are identity-bound and do not change frequently (e.g., specialization, role).
- **Dynamic Attributes:** These are contextual (referred to in research papers as "ContextAttr") and represent situational characteristics that change rapidly (e.g., active shift, patient consent, emergency situation).

The approach I propose for managing these two categories of attributes is as follows:

- Static attributes are securely extracted by the off-chain Gateway and then sent to the Smart Contract.
- Dynamic attributes, on the other hand, are written directly into the PIPSC Smart Contract by the involved actors (Hospital, Patient, or First Responders).
 - o Since the SmartAccess blockchain is a Private and Permissioned network, it is not open to everyone. Every authorized actor already possesses their own cryptographic keys, so there is no need for dynamic (contextual) attributes to be verified and validated by the verifying age

For now, in the example I described, I successfully managed to include and extract the static attributes within the medical degree verification request:

```

91     },
92     "requested_proof": {
93         "revealed_attrs": {
94             "license_id": {
95                 "sub_proof_index": 0,
96                 "raw": "MED-ITA-2026-889900",
97                 "encoded":
98                 "1929902087187012347756855329743386656332003061214443767718922998102139946229"
99             },
100             "professional_role": {
101                 "sub_proof_index": 0,
102                 "raw": "Senior Physician",
103                 "encoded":
104                 "114448090372347274308908214385657783620856542619432597077993171840920873231593"
105             },
106             "medical_specialty": {
107                 "sub_proof_index": 0,
108                 "raw": "Cardiology",
109                 "encoded":
110                 "3445329352963580364409101683082793510676752925719921017949626631811364403336"
111             },
112             "security_clearance": {
113                 "sub_proof_index": 0,
114                 "raw": "Level-3-High",
115                 "encoded":
116                 "83347926831328469786371353227820723321289138820454541784153446959952576676597"
117             }
118         },
119         "self_attested_attrs": {},
120         "unrevealed_attrs": {},
121         "predicates": {}
122     },
123     "identifiers": [
124         {
125             "schema_id": "RcS6Ry9hCj24qEXuq1oxh7:2:Healthcare_Professional_License:2.0",
126             "cred_def_id":
127             "RcS6Ry9hCj24qEXuq1oxh7:3:CL:3153182:official_medical_license_v2",
128             "rev_reg_id": null,
129             "timestamp": null
130         }
131     ]

```

Therefore, the next step will be to automatically read them using a Python script and forward them to the Smart Contracts (PEPSC/PIPSC) to complete the On-Chain access.

To conclude, I present the sequence diagram of the use case just described:

